

LAPORAN PRAKTIKUM
PRAKTIKUM PENGUJIAN PERANGKAT LUNAK

PERTEMUAN 3
Parameterized Testing



Disusun Oleh:

Nama : Abdullah Afif Habiburrohman
NIM : 537611/SV/24441
Kelas : BB / B2
Dosen : Divi Galih Prasetyo Putri, S.Kom., M.Kom., Ph.D.

PROGRAM STUDI SARJANA TERAPAN
TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA

2026

Contents

A Tujuan Pembelajaran	2
B Dasar Teori	2
C Hasil dan Pembahasan	3
C.1 Kode Baru	3
C.2 Perubahan dan penambahan kode	4
C.3 Gambar Hasil Run	9
C.4 GitHub	9
D Kesimpulan	9
E Daftar Pustaka	10

A. Tujuan Pembelajaran

1. Mahasiswa mampu memahami tipe-tipe parameter dalam test method
2. Mahasiswa mampu mengimplementasikan tipe-tipe parameter pada pengujian

B. Dasar Teori

Parameterized test adalah fitur dalam ekosistem JUnit 5 yang memungkinkan eksekusi sebuah metode pengujian secara berulang kali menggunakan serangkaian nilai input atau parameter yang berbeda. Fitur ini memfasilitasi pengujian untuk berbagai kasus dan skenario tanpa mengharuskan pengembang menulis metode pengujian terpisah untuk masing-masing input.

Penerapan pengujian terparameterisasi sangat erat kaitannya dengan prinsip perangkat lunak DRY (Don't Repeat Yourself), di mana redundansi kode dapat ditekan dengan menghindari penyalinan kode pengujian yang identik secara berulang. Selain menjaga agar basis kode pengujian tetap bersih dan mudah dibaca, pendekatan ini juga secara efektif meningkatkan cakupan pengujian (test coverage) untuk berbagai skenario khusus tanpa mengorbankan kemudahan pemeliharaan.

Untuk mengimplementasikan fitur ini, pengujian tidak lagi ditandai dengan anotasi standar, melainkan harus menggunakan anotasi `@ParameterizedTest`. Secara arsitektural, implementasi pengujian ini membutuhkan penambahan dependensi `junit-jupiter-params` pada konfigurasi proyek, dan setiap metode pengujian terparameterisasi diwajibkan untuk mendeklarasikan minimal satu sumber argumen (source of arguments).

JUnit Jupiter menyediakan berbagai macam anotasi sumber untuk menyuplai nilai parameter ke dalam metode pengujian. Beberapa mekanisme utama penyediaan parameter tersebut meliputi:

1. Anotasi `@ValueSource`

Anotasi ini digunakan untuk mendefinisikan array nilai literal tunggal (single-value parameter). Tipe data yang didukung mencakup tipe primitif (seperti `int`, `long`, `double`), `String`, maupun `Class`. JUnit 5 akan menjalankan metode pengujian sebanyak jumlah elemen yang ada di dalam array tersebut, dengan menyuntikkan satu elemen pada setiap eksekusi pengujian. Namun, tipe sumber argumen ini tidak mendukung penyisipan argumen yang bernilai `null`.

2. Anotasi `@CsvSource` dan `@CsvFileSource`

Untuk skenario di mana metode pengujian memerlukan penyisipan lebih dari satu argumen sekaligus (misalnya sepasang nilai input awal dan ekspektasi nilai akhir), sumber data berformat CSV (Comma-Separated Values) dapat dimanfaatkan.

- `@CsvSource` memungkinkan deklarasi deretan parameter secara langsung dalam bentuk sekumpulan string yang dipisahkan oleh tanda koma di dalam anotasi kode sumber.

- @CsvFileSource beroperasi serupa, namun argumen dimuat secara eksternal melalui file CSV yang tersimpan di dalam struktur repositori proyek, sehingga memungkinkan pemisahan yang rapi antara data pengujian dan logika kode.

3. Anotasi @MethodSource

Apabila pengujian membutuhkan pasokan argumen yang direpresentasikan oleh objek yang lebih kompleks, parameter dapat disuplai melalui pemanggilan sebuah metode penyedia (factory method). Metode penyedia parameter ini diwajibkan untuk bersifat statis (static method) dan harus mengembalikan kumpulan argumen dalam format struktur data iteratif, seperti Stream, Iterable, Collection, ataupun sekumpulan array objek.

C. Hasil dan Pembahasan

Mahasiswa melanjutkan praktikum sebelumnya, Wallet dan Testing Lifecycle pada kelas WalletTest. Mahasiswa akan menampilkan kode tambahan dan perubahan kode untuk mengimplementasikan tugas-tugas dalam parameterized test. Berikut adalah kode-kode yang telah diimplementasikan untuk memenuhi tugas praktikum ini:

C.1 Kode Baru

Berikut adalah kode-kode baru yang sebelumnya tidak ada pada praktikum sebelumnya:

```
1 package org.example;
2
3 public class InsufficientFundsException extends Exception {
4     public InsufficientFundsException(String message) {
5         super(message);
6     }
7 }
```

Listing 1: InsufficientFundsException.java

Kode diatas merupakan penambahan exception baru yang akan digunakan untuk mengindikasikan bahwa saldo tidak mencukupi untuk melakukan transaksi tertentu.

```
1 package org.example;
2
3 public class Owner {
4     private final String id;
5     private final String name;
6     private final String email;
7
8     public Owner(String id, String name, String email) {
9         this.id = id;
10        this.name = name;
11        this.email = email;
12    }
13 }
```

```

13
14     public String getId() { return id; }
15     public String getName() { return name; }
16     public String getEmail() { return email; }
17 }

```

Listing 2: Owner.java

Kode diatas adalah kode baru untuk kelas Owner yang sebelumnya hanya string pada kelas Wallet. Kelas Owner ini akan digunakan untuk menyimpan informasi pemilik dompet, seperti id, nama, dan email.

C.2 Perubahan dan penambahan kode

Berikut adalah kode-kode yang telah diubah dan ditambahkan untuk mengimplementasikan tugas-tugas dalam parameterized test:

```

1 package org.example;
2
3 public class Card {
4     private String namaBank;
5     private int nomorRekening;
6
7     public Card(String namaBank, int nomorRekening) {
8         this.namaBank = namaBank;
9         this.nomorRekening = nomorRekening;
10    }
11
12    public String getNamaBank() {
13        return namaBank;
14    }
15
16    public int getNomorRekening() {
17        return nomorRekening;
18    }
19 }

```

Listing 3: Card.java

Kode yang diubah adalah tipe data dari variabel nomorRekening yang sebelumnya berupa String menjadi int. Perubahan ini dilakukan untuk menyesuaikan dengan format nomor rekening yang umumnya berupa angka.

```

1 package org.example;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class Wallet {
7     private Owner owner;
8     private List<Card> cards;
9     private double cash;
10
11    public Wallet(Owner owner) {
12        this.owner = owner;

```

```

13         this.cards = new ArrayList<>();
14         this.cash = 0.0;
15     }
16
17     public void setOwner(Owner owner) {
18         this.owner = owner;
19     }
20
21     public Owner getOwner() {
22         return owner;
23     }
24
25     public void addCards(String bank, int nomor) {
26         Card newCard = new Card(bank, nomor);
27         this.cards.add(newCard);
28     }
29
30     public Card removeCard(int nomor) {
31         for (Card card : cards) {
32             if (card.getNomorRekening() == nomor) {
33                 cards.remove(card);
34                 return card;
35             }
36         }
37         return null;
38     }
39
40     public List<Card> getCards() {
41         return cards;
42     }
43
44     public void deposit(double amount) {
45         if (amount > 0) {
46             this.cash += amount;
47         }
48     }
49
50     public void withdraw(double amount) throws
↪ InsufficientFundsException {
51         if (amount <= 0) {
52             throw new IllegalArgumentException("Withdraw amount
↪ must be > 0");
53         }
54         if (this.cash < amount) {
55             throw new InsufficientFundsException("Insufficient
↪ funds");
56         }
57         this.cash -= amount;
58     }
59
60     public double getCash() {
61         return cash;
62     }
63
64     public void cleanCash() {
65         this.cash = 0.0;

```

```

66     }
67
68     public void cleanCards() {
69         cards.clear();
70     }
71 }

```

Listing 4: Wallet.java

Kode diatas adalah kode yang telah diubah dan ditambahkan pada kelas Wallet. Perubahan utama adalah:

1. Mengubah owner dan turunannya dari tipe String menjadi tipe Owner untuk menyimpan informasi pemilik dompet secara lebih terstruktur.
2. Mengubah tipe data variabel cash menjadi double untuk memungkinkan penyimpanan nilai uang dengan desimal.
3. Mengimplementasikan perubahan tipe data variabel nomorRekening dari String menjadi int pada kelas Card ke kelas Wallet.
4. Mengubah metode withdraw menjadi melemparkan `InsufficientFundsException` jika jumlah penarikan melebihi saldo yang tersedia, serta menambahkan validasi untuk memastikan jumlah penarikan harus lebih besar dari 0.

```

1 package org.example;
2
3 import org.junit.jupiter.api.*;
4 import org.junit.jupiter.params.ParameterizedTest;
5 import org.junit.jupiter.params.provider.Arguments;
6 import org.junit.jupiter.params.provider.CsvFileSource;
7 import org.junit.jupiter.params.provider.MethodSource;
8 import org.junit.jupiter.params.provider.ValueSource;
9
10 import java.util.stream.Stream;
11
12 import static org.junit.jupiter.api.Assertions.*;
13
14 @TestInstance(TestInstance.Lifecycle.PER_CLASS)
15 @TestMethodOrder(MethodOrderer.OrderAnnotation.class)
16 class WalletTest {
17
18     Wallet dompet;
19
20     @BeforeAll
21     void initClass() {
22         Owner defaultOwner = new Owner("1", "Abdullah", "
↪ abd@ugm.ac.id");
23         dompet = new Wallet(defaultOwner);
24     }
25
26     @AfterAll
27     void cleanClass() {
28         dompet = null;
29     }
30

```

```

31     @BeforeEach
32     void initMethod() {
33         dompet.deposit(10000.0);
34         dompet.addCards("DefaultBank", 0);
35     }
36
37     @AfterEach
38     void cleanMethod() {
39         dompet.cleanCards();
40         dompet.cleanCash();
41     }
42
43     @Test
44     @Order(1)
45     @DisplayName("Execute constructor, verify fields")
46     void testIsWallet() {
47         assertNotNull(dompet, "Objek dompet tidak boleh null");
48         assertEquals("Abdullah", dompet.getOwner().getName());
49     }
50
51     @ParameterizedTest
52     @ValueSource(doubles = {10000.0, 50000.0, 100000.0})
53     @DisplayName("Tugas 1: Test Nominal Cash Valid (Deposit)")
54     void testCashValid(double nominal) {
55         dompet.cleanCash();
56         dompet.deposit(nominal);
57         assertEquals(nominal, dompet.getCash());
58     }
59
60     @ParameterizedTest
61     @ValueSource(doubles = {-1000.0, -5000.0, 0.0})
62     @DisplayName("Tugas 1: Test Nominal Cash Tidak Valid (
↪ Deposit diabaikan)")
63     void testCashInvalid(double nominal) {
64         dompet.cleanCash();
65         dompet.deposit(nominal);
66         assertEquals(0.0, dompet.getCash()); // Saldo tetap 0
67     }
68
69     @ParameterizedTest
70     @CsvFileSource(resources = "/valid-withdraw.csv",
↪ numLinesToSkip = 1)
71     @DisplayName("Tugas 2: Test Withdraw Valid (CSV)")
72     void testValidWithdrawCSV(double deposit, double withdraw,
↪ double expectedTotal) throws InsufficientFundsException {
73         dompet.cleanCash();
74         dompet.deposit(deposit);
75
76         if (withdraw > 0) {
77             dompet.withdraw(withdraw);
78         }
79
80         assertEquals(expectedTotal, dompet.getCash());
81     }
82
83     @ParameterizedTest

```



```

84     @CsvFileSource(resources = "/invalid-withdraw.csv",
    ↪ numLinesToSkip = 1)
85     @DisplayName("Tugas 2: Test Withdraw Invalid Exception (CSV
    ↪ )")
86     void testInvalidWithdrawCSV(double deposit, double withdraw
    ↪ , String exceptionType) {
87         dompet.cleanCash();
88         dompet.deposit(deposit);
89
90         if (exceptionType.equals("InsufficientFundsException"))
    ↪ {
91             assertThrows(InsufficientFundsException.class, ()
    ↪ -> dompet.withdraw(withdraw));
92         } else if (exceptionType.equals("
    ↪ IllegalArgumentException")) {
93             assertThrows(IllegalArgumentException.class, () ->
    ↪ dompet.withdraw(withdraw));
94         }
95     }
96
97     static Stream<Arguments> provideOwnerObjects() {
98         return Stream.of(
99             Arguments.of(new Owner("1", "Agus", "agus@ugm.
    ↪ ac.id")),
100        Arguments.of(new Owner("2", "Siti", "siti@ugm.
    ↪ ac.id"))
101     );
102 }
103
104 @ParameterizedTest
105 @MethodSource("provideOwnerObjects")
106 @DisplayName("Tugas 3: Test Set Owner Menggunakan Object")
107 void testSetOwnerObject(Owner ownerParam) {
108     dompet.setOwner(ownerParam);
109     assertNotNull(dompet.getOwner());
110     assertEquals(ownerParam.getName(), dompet.getOwner().
    ↪ getName());
111     assertEquals(ownerParam.getEmail(), dompet.getOwner().
    ↪ getEmail());
112 }
113 }

```

Listing 5: WalletTest.java

Kode diatas adalah kode yang telah diubah dan ditambahkan pada kelas WalletTest. Perubahan utama adalah:

1. Mengimplementasikan perubahan pada class Wallet yang telah dilakukan sebelumnya.
2. Menambahkan @ParameterizedTest pada metode testCashValid dan testCashInvalid dengan value source berupa array double untuk menguji berbagai nominal deposit yang valid dan tidak valid.
3. Menambahkan @ParameterizedTest pada metode testValidWithdrawCSV dan

testInvalidWithdrawCSV dengan value source berupa file CSV untuk menguji berbagai kasus withdraw yang valid dan tidak valid.

4. Menambahkan @ParameterizedTest pada metode testSetOwnerObject dengan value source berupa objek Owner untuk menguji berbagai kasus owner yang valid.

C.3 Gambar Hasil Run

Berikut adalah gambar hasil run dari kode diatas:

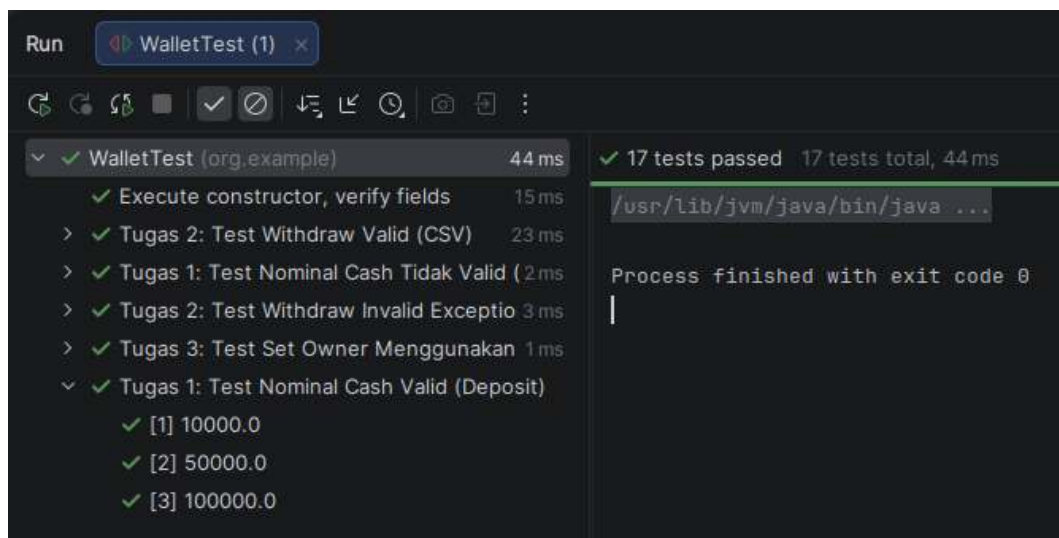


Figure 1: Hasil Run

C.4 GitHub

Berikut adalah link repository di GitHub yang berisi kode diatas <https://github.com/bukunya/PPPL/tree/Pertemuan3>

D. Kesimpulan

Pada praktikum ini mahasiswa telah mempelajari dan mengimplementasikan konsep parameterized testing menggunakan JUnit 5. Dengan berbagai sumber data seperti @ValueSource, file CSV (@CsvFileSource), dan @MethodSource, pengujian dapat dijalankan berkali-kali dengan berbagai input tanpa mengulang kode.

Beberapa hal penting yang diperoleh dari praktikum:

- Penggunaan anotasi @ParameterizedTest memudahkan pengujian banyak kasus.
- Sumber data fleksibel (literal, CSV, objek) mendukung skenario pengujian yang kompleks.

- Penanganan exception seperti `InsufficientFundsException` dan validasi `IllegalArgumentException` dapat diuji secara terstruktur.
- Refaktor kelas pendukung (`Owner`, `Card`, `Wallet`) meningkatkan keterbacaan dan kesiapan untuk menerima parameter terperinci.

Secara keseluruhan, praktikum ini berhasil memenuhi tujuan pembelajaran dengan menunjukkan bahwa metode pengujian terparameterisasi efektif untuk meningkatkan cakupan dan meminimalkan duplikasi. Mahasiswa diharapkan dapat menerapkan pendekatan serupa pada proyek pengujian berikutnya.

E. Daftar Pustaka

- [1] Medium - All You Need to Know About Parameterized Tests <https://medium.com/@torresgmarina/all-you-need-to-know-about-parameterized-tests-493af4705508>
- [2] JUnit - Parameterized Classes and Tests <https://docs.junit.org/6.0.3/writing-tests/parameterized-classes-and-tests.html>
- [3] Baeldung - Parameterized Tests in JUnit 5 <https://www.baeldung.com/parameterized-tests-junit-5>